



DoubleML

DoubleML

Hyperparameter Tuning with Optuna

DoubleML Workshop

February 27 & March 06, 2026

Economic AI

Motivation

Why Hyperparameter Tuning?

- **High-quality ML** is one of the three key ingredients of Double Machine Learning
- The predictive performance of nuisance learners directly affects:
 - Bias of causal estimates
 - Standard errors of causal estimates
 - Width of confidence intervals
- Default hyperparameters rarely optimal for specific datasets

Goal: Find hyperparameter configurations that minimize prediction error for nuisance functions (typically one outcome model g_0 and treatment model m_0)

- **DoubleML** provides integrated tuning via Optuna

Basic Tuning Example

Setup: Data and Learners

► Imports & Data

Nuisance Learners

- For this example, we use LightGBM learners
- Initialize a DoubleMLIRM model with default hyperparameters

```
1 ml_g = LGBMRegressor(random_state=314, verbose=-1)
2 ml_m = LGBMClassifier(random_state=314, verbose=-1)
3
4 dml_obj_untuned = DoubleMLIRM(
5     dml_data,
6     ml_g,
7     ml_m,
8 )
9
10 dml_obj_untuned.fit()
11 dml_obj_untuned.summary
```

	coef	std err	t	P> t	2.5 %	97.5 %
d	0.062116	0.877755	0.070783	0.943571	-1.657852	1.782083

Defining Parameter Spaces

- Parameter spaces are defined as **callables** using Optuna's **trial** object

```
1 # Parameter space for outcome regression
2 def ml_g_params(trial):
3     return {
4         'n_estimators': 100, # fixed to keep execution of the slides and simple
5         'learning_rate': trial.suggest_float('learning_rate', 0.001, 0.1, log=True),
6         'min_child_samples': trial.suggest_int('min_child_samples', 20, 50, step=10),
7         'lambda_l1': trial.suggest_float('lambda_l1', 1e-2, 10.0, log=True),
8         'lambda_l2': trial.suggest_float('lambda_l2', 1e-2, 10.0, log=True),
9     }
10
11 # Parameter space for propensity score
12 def ml_m_params(trial):
13     return {
14         'n_estimators': 100, # fixed to keep execution of the slides and simple
15         'learning_rate': trial.suggest_float('learning_rate', 0.001, 0.1, log=True),
16         'min_child_samples': trial.suggest_int('min_child_samples', 20, 50, step=10),
17         'lambda_l1': trial.suggest_float('lambda_l1', 1e-2, 10.0, log=True),
18         'lambda_l2': trial.suggest_float('lambda_l2', 1e-2, 10.0, log=True),
19     }
```

Optuna Search Space Types

Method	Description	Example
<code>suggest_float</code>	Continuous values	<code>trial.suggest_float('lr', 0.001, 0.1, log=True)</code>
<code>suggest_int</code>	Integer values	<code>trial.suggest_int('depth', 2, 10)</code>
<code>suggest_categorical</code>	Categorical choices	<code>trial.suggest_categorical('kernel', ['rbf', 'linear'])</code>

- Use `log=True` for parameters that vary over orders of magnitude
- Use `step` for discrete increments

Tuning Hyperparameters

```
1 param_space = {
2     'ml_g': ml_g_params,
3     'ml_m': ml_m_params
4 }
5
6 # Optuna settings
7 optuna_settings = {
8     'n_trials': 100,
9     'show_progress_bar': True,
10    'verbosity': optuna.logging.WARNING,
11 }
12
13 # Initialize and tune
14 dml_obj_tuned = DoubleMLIRM(dml_data, ml_g, ml_m)
15
16 dml_obj_tuned.tune_ml_models(
17     ml_param_space=param_space,
18     optuna_settings=optuna_settings,
19 )
```

Best trial: 83. Best value: -1.01297: 100%

100/100 [00:04<00:00, 22.69it/s]

Best trial: 96. Best value: -1.06852: 100%

100/100 [00:03<00:00, 20.94it/s]

Best trial: 96. Best value: -0.495723: 100%

100/100 [00:06<00:00, 13.89it/s]

<doubleml.irm.irm.DoubleMLIRM at 0x7f73a75f5250>

Fit After Tuning

- Best hyperparameters are set automatically
- Proceed with standard `fit()` call

1 dml_obj_tuned.fit()
2 dml_obj_tuned.summary

	coef	std err	t	P> t	2.5 %	97.5 %
d	0.464453	0.148052	3.137095	0.001706	0.174276	0.754629

Understanding `params_names`

- Some models internally create multiple learners from `ml_g` and `ml_m`

```
1 dml_obj_tuned.params_names
```

```
['ml_g0', 'ml_g1', 'ml_m']
```

- `ml_g0`: Outcome model for control group ($D=0$)
- `ml_g1`: Outcome model for treatment group ($D=1$)
- `ml_m`: Propensity score model

Each is tuned separately with the same parameter space!

Inspecting Tuned Parameters

- Identical hyperparameters are set for each cross-fitting fold

```
1 dml_obj_tuned.params
```

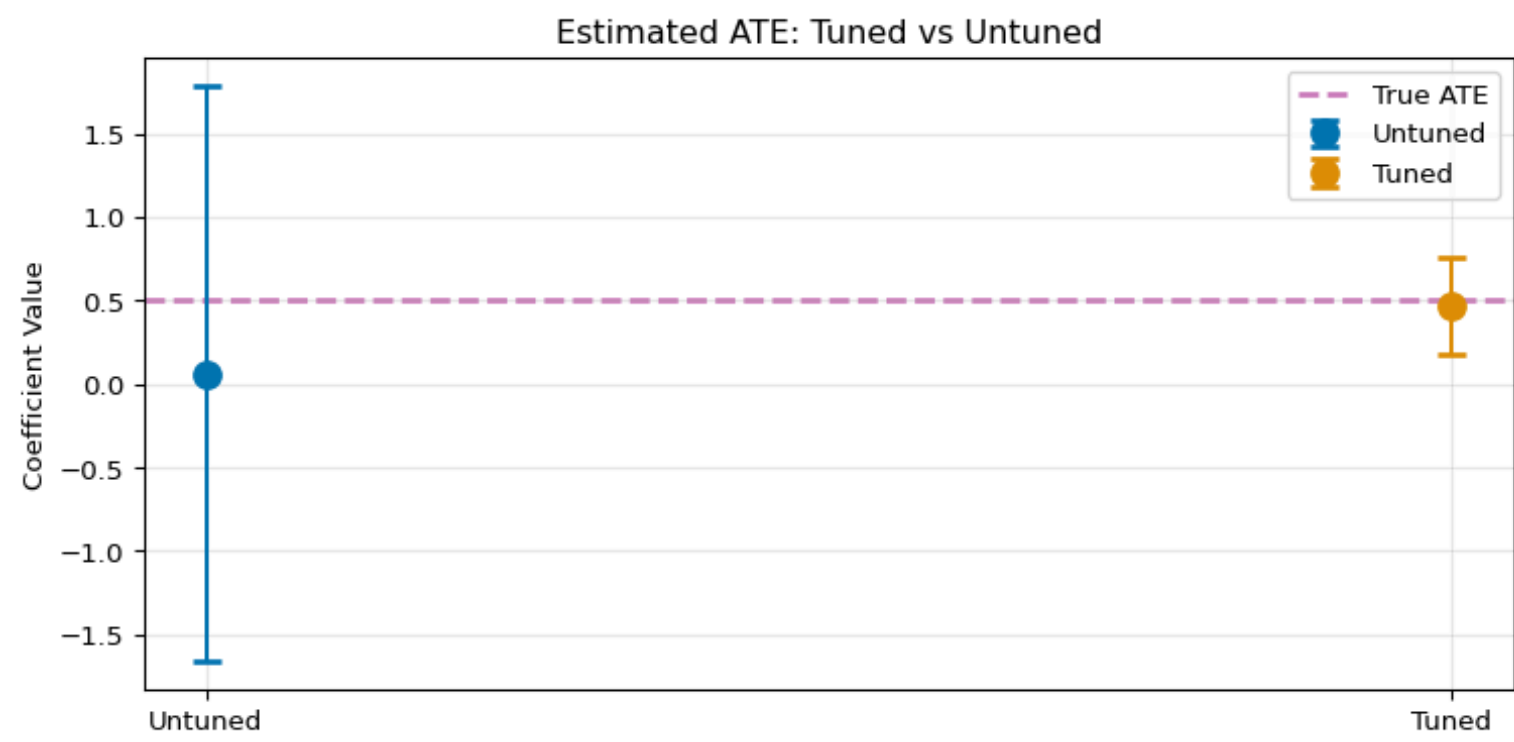
```
{'ml_g0': {'d': [[{'learning_rate': 0.001658993264989866,  
  'min_child_samples': 20,  
  'lambda_l1': 6.98824424288107,  
  'lambda_l2': 0.056404445951061366},  
{'learning_rate': 0.001658993264989866,  
  'min_child_samples': 20,  
  'lambda_l1': 6.98824424288107,  
  'lambda_l2': 0.056404445951061366},  
{'learning_rate': 0.001658993264989866,  
  'min_child_samples': 20,  
  'lambda_l1': 6.98824424288107,  
  'lambda_l2': 0.056404445951061366},  
{'learning_rate': 0.001658993264989866,  
  'min_child_samples': 20,  
  'lambda_l1': 6.98824424288107,  
  'lambda_l2': 0.056404445951061366},  
{'learning_rate': 0.001658993264989866,  
  'min_child_samples': 20,  
  'lambda_l1': 6.98824424288107,  
  'lambda_l2': 0.056404445951061366}]]},  
...
```

Comparison: Tuned vs Untuned

► Code

True Treatment Effect: 0.5					
Model	theta	se	ci_lower	ci_upper	
Untuned	0.062116	0.877550	-1.657852	1.782083	
Tuned	0.464453	0.148052	0.174276	0.754629	

► Code



Evaluating Learner Performance

- The evaluation of the nuisance learners is crucial for understanding model performance
- This can be done via `evaluate_learners()` or accessing the predictions directly

```
1 print("Untuned:")
2 print(dml_obj_untuned.evaluate_learners())
```

```
Untuned:
{'ml_g0': array([[1.1162421]]), 'ml_g1': array([[1.20162817]]), 'ml_m': array([[0.45040263]])}
```

```
1 print("\nTuned:")
2 print(dml_obj_tuned.evaluate_learners())
```

```
Tuned:
{'ml_g0': array([[1.01241188]]), 'ml_g1': array([[1.09037055]]), 'ml_m': array([[0.41151658]])}
```

Advanced Tuning with Pipelines

Using Sklearn Pipelines

- Complex learners can be constructed using **Pipeline** and/or stacked learners via **StackingRegressor** or **StackingClassifier**

► Imports & Classification Pipeline

```
1 # Define base regressors
2 base_regressors = [
3     ('linear_regression', LinearRegression()),
4     ('lgbm', LGBMRegressor(random_state=42, verbose=-1))
5 ]
6
7 # Create stacking regressor
8 stacking_regressor = StackingRegressor(
9     estimators=base_regressors,
10    final_estimator=Ridge()
11 )
12
13 # Build pipeline
14 ml_g_pipeline = Pipeline([
15     ('scaler', RobustScaler()),
16     ('stacking', stacking_regressor)
17 ])
```

Parameter Spaces for Pipelines

- Use double underscore `__` notation to access nested parameters
- You can set parameter space for different levels of the pipeline

```
1 def ml_g_params_pipeline(trial):  
2     return {  
3         'stacking__lgbm__n_estimators': 100,  
4         'stacking__lgbm__learning_rate': trial.suggest_float(  
5             'stacking__lgbm__learning_rate', 0.001, 0.1, log=True),  
6         'stacking__lgbm__max_depth': 5,  
7         'stacking__lgbm__lambda_l1': trial.suggest_float(  
8             'stacking__lgbm__lambda_l1', 1e-3, 10.0, log=True),  
9         'stacking__final_estimator__alpha': trial.suggest_float(  
10            'stacking__final_estimator__alpha', 0.001, 10.0, log=True),  
11     }
```

Learner-Specific Settings

- Override global settings for specific learners

```
1 optuna_settings_pipeline = {  
2     'n_trials': 100,  
3     'show_progress_bar': True,  
4     'verbosity': optuna.logging.WARNING,  
5     'ml_g': {  
6         'n_trials': 50 # Fewer trials for ml_g  
7     }  
8 }
```

- Settings can be specified at **learner_names** level (**ml_g**, **ml_m**) or **params_names** level (**ml_g_d_lvl0**, **ml_g_d_lvl1**)
- More specific settings override general settings

Analyzing Tuning Results

Returning Tuning Results

► Parameter Space

```
1 dml_obj_tuned_pipeline = DoubleMLIRM(  
2     dml_data, ml_g_pipeline, ml_m_pipeline  
3 )  
4  
5 tuning_results = dml_obj_tuned_pipeline.tune_ml_models(  
6     ml_param_space=param_space_pipeline,  
7     optuna_settings=optuna_settings_pipeline,  
8     return_tune_res=True, # Return detailed results  
9 )  
10  
11 dml_obj_tuned_pipeline.fit()  
12 dml_obj_tuned_pipeline.summary
```

Best trial: 16. Best value: -1.01094: 100% 50/50 [00:11<00:00, 4.44it/s]

Best trial: 49. Best value: -1.06139: 100% 50/50 [00:09<00:00, 5.33it/s]

Best trial: 93. Best value: -0.50215: 100% 100/100 [00:47<00:00, 2.17it/s]

	coef	std err	t	P> t	2.5 %	97.5 %
d	0.520423	0.139378	3.733889	0.000189	0.247246	0.793599

Accessing Optuna Studies

- Detailed tuning results and studies are available as a dictionary

```
1 # Available tuning results
2 print(tuning_results[0].keys())
```

```
dict_keys(['ml_g0', 'ml_g1', 'ml_m'])
```

```
1 # Detailed results for ml_m
2 print(tuning_results[0]['ml_m'])
```

```
===== DMLOptunaResult =====
```

```
Learner name: ml_m
```

```
Params name: ml_m
```

```
Tuned: True
```

```
Best score: -0.5021496582916085
```

```
Scoring method: neg_log_loss
```

```
----- Best parameters -----
```

```
{'stacking__final_estimator__C': 28.26087394481712,
 'stacking__lgbm__lambda_l1': 3.241800482741334,
 'stacking__lgbm__lambda_l2': 0.017315675287715316,
 'stacking__lgbm__learning_rate': 0.03677412502163472,
 'stacking__lgbm__min_child_samples': 20}
```

Trials DataFrame

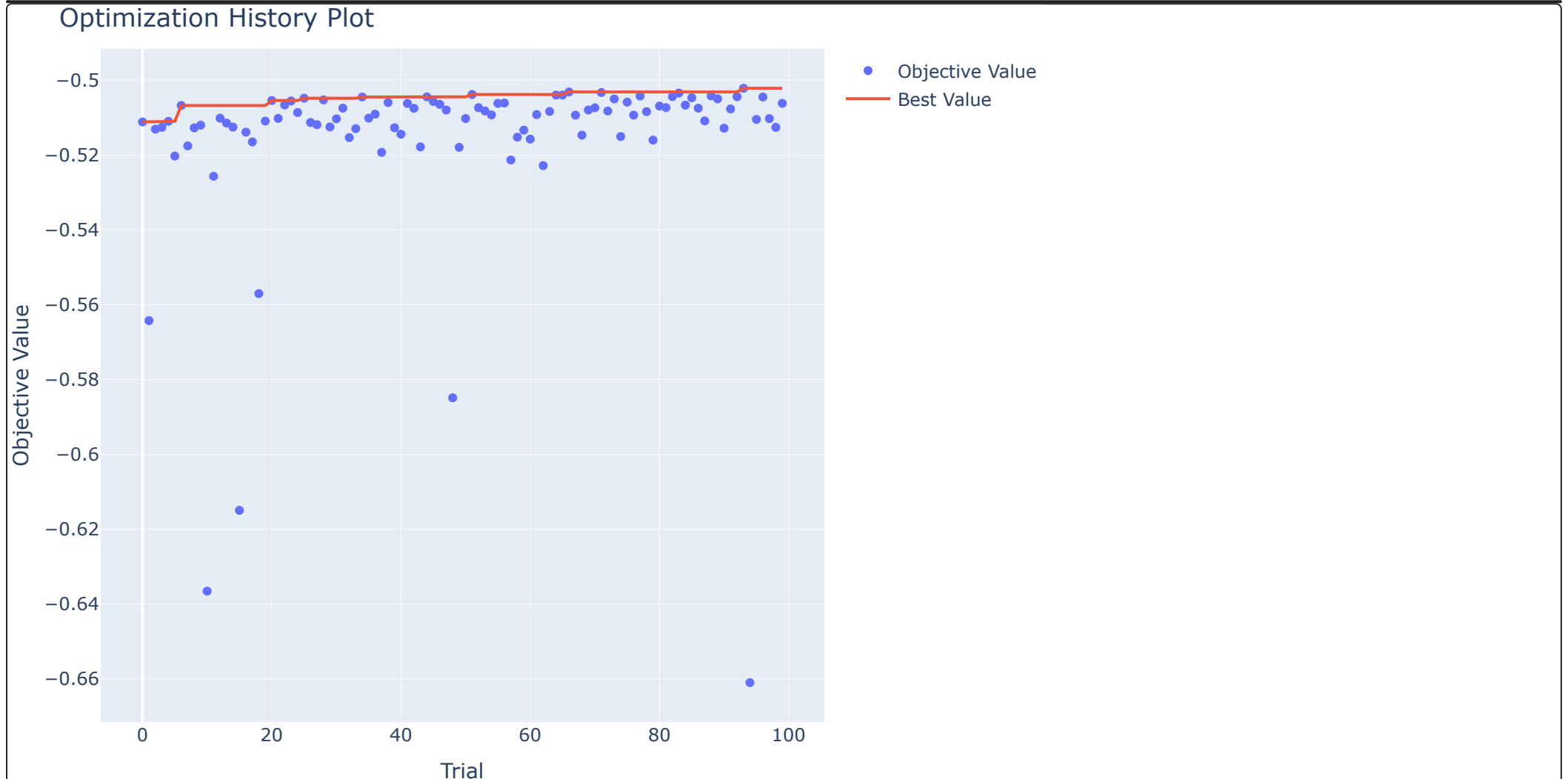
- The optuna study object includes informations on each trial run

```
1 ml_m_study = tuning_results[0]['ml_m'].study
2 ml_m_study.trials_dataframe().head()
```

	number	value	datetime_start	datetime_complete	duration	param
0	0	-0.511154	2026-02-27 08:39:20.671080	2026-02-27 08:39:21.138010	0 days 00:00:00.466930	1.56174
1	1	-0.564255	2026-02-27 08:39:21.139294	2026-02-27 08:39:21.494414	0 days 00:00:00.355120	0.13909
2	2	-0.513081	2026-02-27 08:39:21.495326	2026-02-27 08:39:22.122630	0 days 00:00:00.627304	22.1089
3	3	-0.512590	2026-02-27 08:39:22.123283	2026-02-27 08:39:22.557234	0 days 00:00:00.433951	2.28567
4	4	-0.510994	2026-02-27 08:39:22.558037	2026-02-27 08:39:22.992345	0 days 00:00:00.434308	14.2491

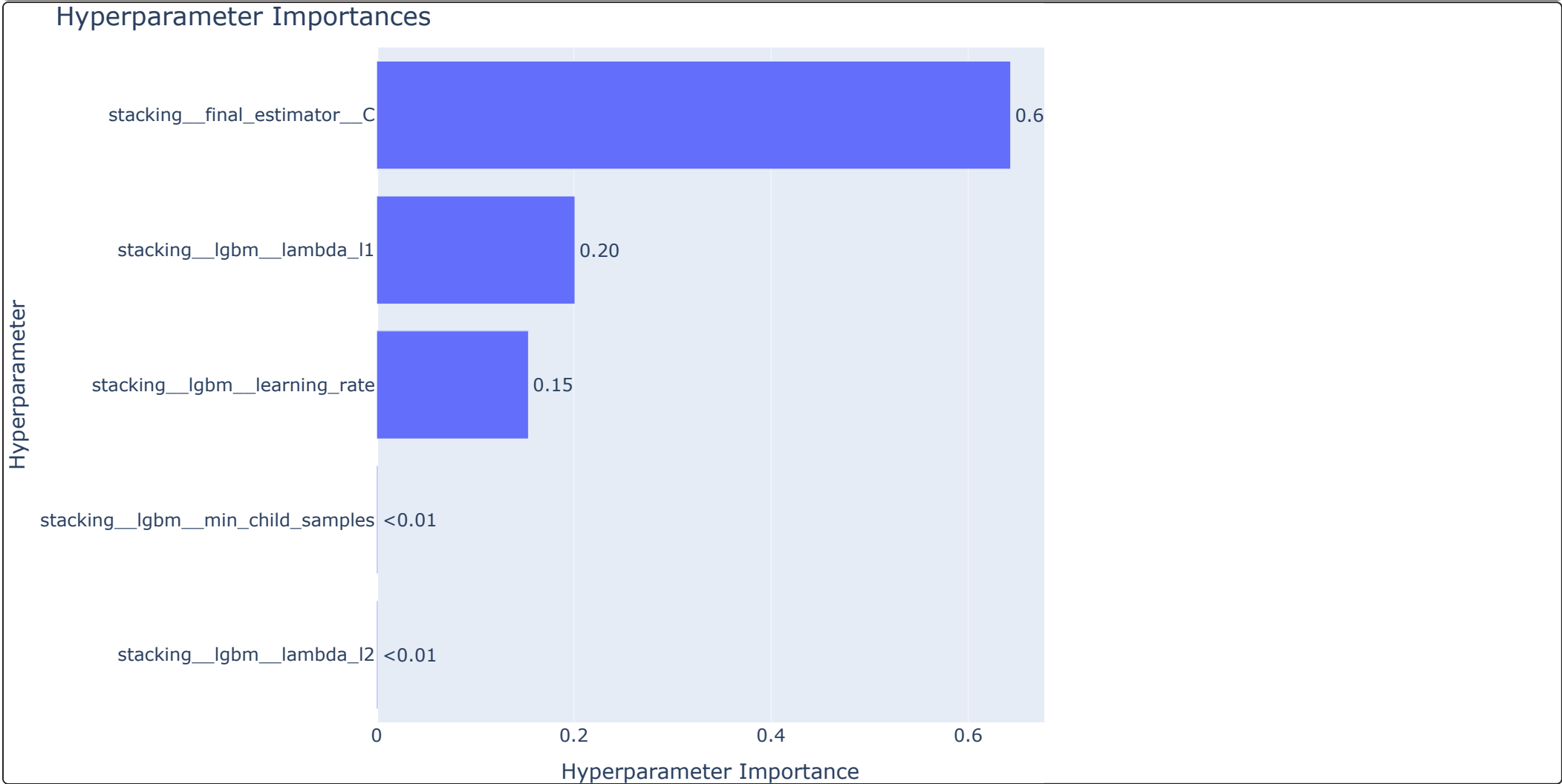
Optuna Visualization: Optimization History

```
1 from plotly.io import show
2
3 fig = optuna.visualization.plot_optimization_history(ml_m_study)
4 show(fig)
```



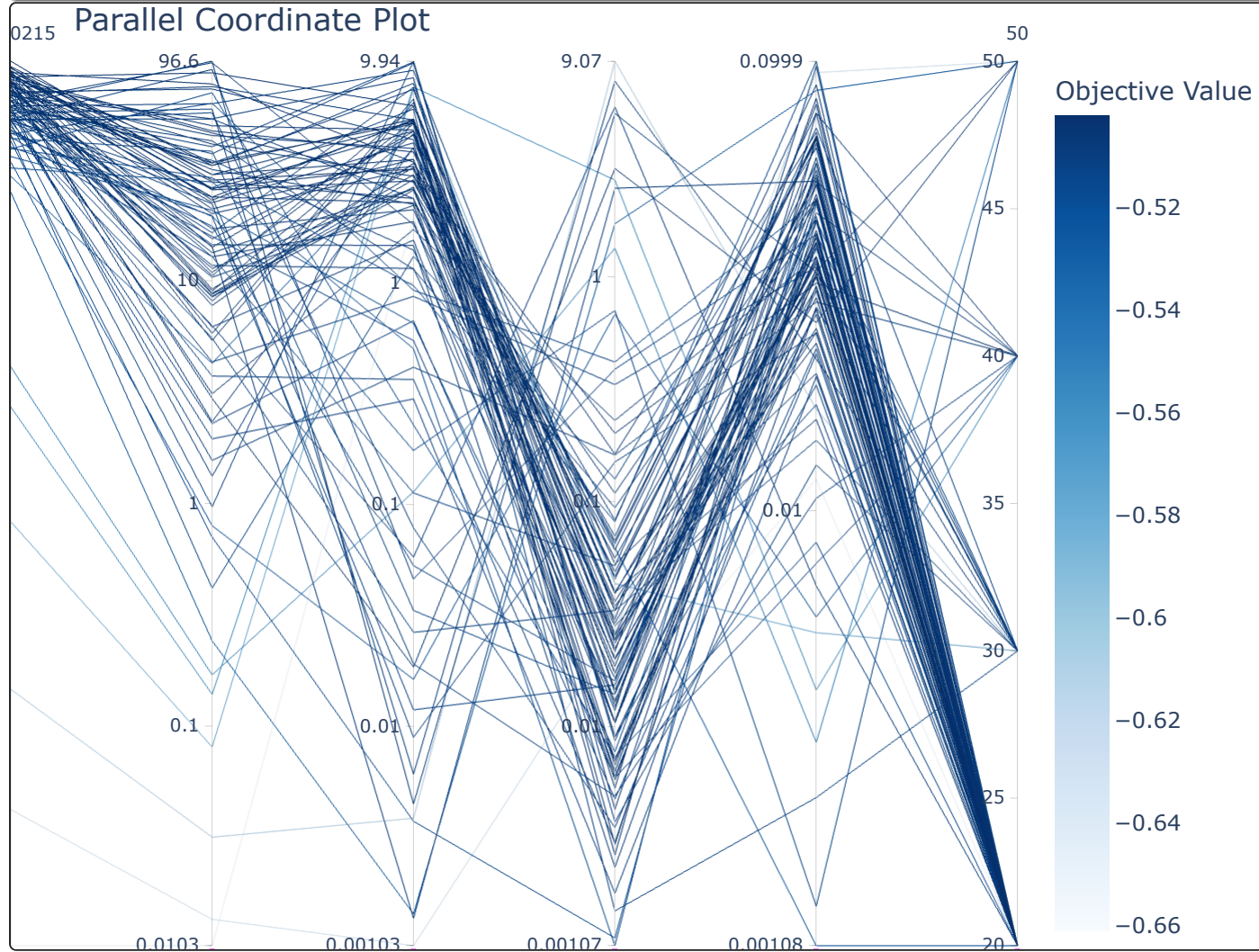
Optuna Visualization: Parameter Importance

```
1 fig = optuna.visualization.plot_param_importances(ml_m_study)
2 show(fig)
```



Optuna Visualization: Parallel Coordinates

```
1 fig = optuna.visualization.plot_parallel_coordinate(ml_m_study)
2 show(fig)
```



Summary

Key Takeaways

- `tune_ml_models()` provides integrated Optuna tuning
- Define parameter spaces as callables
- Use `optuna_settings` for configuration
- Analyze results with Optuna visualizations

Resources

- [DoubleML Documentation](#)
- [Optuna Documentation](#)
- [Example Gallery](#)

Appendix

Optuna Settings Reference

Setting	Description	Default
<code>n_trials</code>	Number of optimization trials	100
<code>show_progress_bar</code>	Display progress	False
<code>verbosity</code>	Logging level	<code>WARNING</code>
<code>sampler</code>	Sampling strategy	<code>TPESampler</code>
<code>pruner</code>	Early stopping strategy	<code>MedianPruner</code>

Copyright



Economic AI GmbH
Nürnberger Str. 262 A
93059 Regensburg
Germany

info@economicaai.com

<https://www.economicaai.com>

Copyright ©2024 Economic AI GmbH. No part of this publication may be reproduced, stored in a retrieval system, used in a spreadsheet or document, or transmitted in any form or by any means electronic, mechanical, photocopying, recording, or otherwise without the permission of Economic AI.

References

- Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A Next-generation Hyperparameter Optimization Framework. *KDD*.
- Bach, P., Chernozhukov, V., Kurz, M. S., & Spindler, M. (2022). DoubleML - An Object-Oriented Implementation of Double Machine Learning in Python. *JMLR*.

Bach, Philipp, Victor Chernozhukov, Malte S Kurz, and Martin Spindler. 2022. “DoubleML-an Object-Oriented Implementation of Double Machine Learning in Python.” *Journal of Machine Learning Research* 23: 53–51.

Bach, Philipp, Malte S. Kurz, Victor Chernozhukov, Martin Spindler, and Sven Klaassen. 2024. “DoubleML: An Object-Oriented Implementation of Double Machine Learning in R.” *Journal of Statistical Software*. <https://doi.org/10.18637/jss.v108.i03>.

Chernozhukov, Victor, Christian Hansen, Nathan Kallus, Martin Spindler, and Vasilis Syrgkanis. 2024. *Applied Causal Inference Powered by Machine Learning and AI*. CausalML-book.org. <https://causalml-book.org>.